



Université du Québec
à Chicoutimi

Jalon 3

8INF970 - Atelier pratique en cybersécurité II

Fehmi Jaafar Samuel Desbiens

Justin Bossard

Mattéo Gouhier

Paul Mathé

Samuel Plet

Léo Raclet

5 avril 2026

Table des matières

1	Introduction	2
2	Évolution vers une Architecture Hybride (RAG & LLM)	3
2.1	Changement de Paradigme : De RoBERTa à Llama 3	3
2.2	Abandon du Fine-Tuning au profit du Prompt Engineering	3
2.3	Le Système de Mémoire : Architecture RAG	4
2.3.1	1. Scraping RSS	4
2.3.2	2. Vectorisation et Stockage (ChromaDB)	4
2.3.3	3. Prompt Enrichi	5
2.4	Nouveau Format de Sortie (Standard JSON v2)	5
2.5	Flux final	5
2.5.1	1. Entrée	5
2.5.2	2. Récupération	5
2.5.3	3. Travail du LLM	5
2.5.4	4. Sortie	5
2.6	Sécurité et Robustesse Technique	5
3	Conclusion	6
	Références	7

Partie 1

Introduction

Partie 2

Évolution vers une Architecture Hybride (RAG & LLM)

Pour dépasser les limites d’une simple classification “Vrai/Faux”, l’architecture a évolué vers un système de **Génération Augmentée par Récupération (RAG)**. Cette approche permet non seulement de détecter la désinformation, mais aussi de fournir une justification sourcée en se basant sur des faits réels et actualisés.

2.1 Changement de Paradigme : De RoBERTa à Llama 3

Initialement basé sur RoBERTa pour la classification, le système utilise désormais **Llama 3**. Ce changement est motivé par deux besoins :

- **Le Raisonnement** : Capacité à expliquer *pourquoi* une information est jugée suspecte. RoBERTa ne donnait qu’une réponse Vrai/Faux, mais Llama, en tant que LLM, permet de justifier le choix.
- **Le Contexte** : Capacité à comprendre le contexte global de l’article, et pas seulement une phrase isolée.

2.2 Abandon du Fine-Tuning au profit du Prompt Engineering

La stratégie initiale envisageait un fine-tuning supervisé (SFT) sur un jeu de données spécifique. Cette approche a été écartée au profit du *Prompt Engineering* couplé à l’architecture RAG, en raison de plusieurs contraintes techniques et méthodologiques.

Bien que non retenu pour l’itération actuelle, le fine-tuning supervisé présente des avantages documentés. En termes de performance et de précision, comme le démontrent Shin et al. (2025), les modèles fine-tunés peuvent surpasser significativement les approches de prompt engineering (jusqu’à 28 points sur certains benchmarks), offrant une stabilité supérieure pour des tâches hautement spécialisées. De plus, l’entraînement permet de contraindre le modèle à générer systématiquement une structure JSON exacte, limitant ainsi les erreurs de syntaxe. Cette méthode optimise également l’inférence en internalisant les règles de formatage, ce qui réduit le nombre de tokens nécessaires en entrée et diminue le temps d’apparition du premier mot (*Time To First Token*). Enfin, les méthodes PEFT (comme LoRA) permettent d’obtenir des performances élevées sur des modèles de taille modeste tout en réduisant les coûts de calcul, offrant un compromis optimal entre précision et ressources (Vangibhurathachhi 2025).

Cependant, la mise en œuvre de cette méthode s’est heurtée à la difficulté de constituer un ensemble de données adéquat. Trois sources principales avaient été identifiées initialement : Webz.io (un répertoire de contenus thématiques), ISOT (un jeu de données binaire d’articles réels et factices) et FakeNewsNet

(un dépôt de données vérifiées de PolitiFact et GossipCop). Pour générer un dataset performant, il est impératif de coupler chaque article à son verdict et à une justification textuelle précise. Or, ces bases manquent de la justification native nécessaire. L’exploitation de Webz.io a été écartée en raison d’un volume brut excédant les ressources matérielles et de l’absence de verdicts validés par des experts. FakeNewsNet, malgré ses labels clairs, souffre quant à lui d’une absence totale de justifications textuelles.

Puisque le fine-tuning exige des ressources importantes et des données volumineuses de haute qualité, l’absence de justifications natives aurait rendu nécessaire l’usage de données synthétiques. Cette alternative introduit des risques critiques de biais. D’une part, la généralisation abusive via des justifications stéréotypées peut limiter le modèle à reproduire des patrons de phrases au lieu d’analyser le texte. D’autre part, forcer la génération de justifications complexes sans lien direct avec les faits réels crée une incohérence sémantique qui augmente drastiquement le risque d’introduire des hallucinations dans le dataset. Par ailleurs, l’usage d’un dataset restreint forcerait une mémorisation d’exemples spécifiques (surapprentissage ou *overfitting*) au détriment de la logique de détection universelle.

Outre la problématique des données, le recours au fine-tuning s’est avéré être une démarche superflue face aux capacités des modèles récents. Le système repose désormais sur Llama 3 (version Instruct), un modèle nativement optimisé pour suivre des directives complexes et produire des formats de sortie stricts comme le JSON. Réentraîner les poids d’un modèle de plusieurs milliards de paramètres uniquement pour lui imposer une structure de réponse représente un effort disproportionné par rapport aux gains attendus.

Par conséquent, le passage au *Prompt Engineering* et au RAG a été privilégié pour la flexibilité accrue qu’il offre. Comme le soulignent Kermani et al. (2025), le prompt engineering et le RAG permettent une adaptation rapide et un déploiement plus flexible, particulièrement adaptés aux scénarios nécessitant une généralisation. Cette approche facilite également la gestion des ressources ; Pornprasit et Tantithamthavorn (2024) recommandent l’approche *few-shot* lorsque les données sont insuffisantes pour un fine-tuning complet, permettant une performance modérée avec une consommation de ressources réduite. De plus, elle assure une séparation claire des préoccupations en intégrant des informations externes actuelles via une base vectorielle (ChromaDB). Comme le confirment Ovadia et al. (2024), l’injection de connaissances par récupération (RAG) est plus adaptée que le fine-tuning pour gérer des données dynamiques, laissant ainsi le modèle se concentrer sur l’analyse logique. Enfin, la maintenance est grandement simplifiée, car l’ajustement des prompts en temps réel évite les cycles d’entraînement chronophages et la lourde gestion de fichiers de modèles.

2.3 Le Système de Mémoire : Architecture RAG

L’innovation majeure réside dans l’ajout d’une “mémoire factuelle” qui permet à l’IA de consulter des sources de confiance avant de répondre.

2.3.1 1. Scraping RSS

Le système n’est plus statique. Un module de collecte automatisé (`scripts/fetch_news.py`) s’exécute tout les jours pour parcourir les flux **RSS** de sources de presse reconnues (Le Monde et France 24). Grâce aux bibliothèques **Feedparser** et **Newspaper3k**, le contenu textuel est extrait, nettoyé de ses balises HTML, et centralisé dans un dataset de référence.

2.3.2 2. Vectorisation et Stockage (ChromaDB)

Pour que l’IA puisse “chercher” dans ces milliers d’articles, nous utilisons :

- **Sentence-Transformers (all-MiniLM-L6-v2)** : Ce modèle transforme chaque article de presse en un vecteur mathématique de 384 dimensions représentant son sens sémantique.

- **ChromaDB** : Une base de données vectorielle haute performance qui stocke ces vecteurs. Elle permet de trouver rapidement les articles les plus proches sémantiquement de la requête de l'utilisateur.

2.3.3 3. Prompt Enrichi

Maintenant, avant chaque requête, on récupère l'article de notre base le plus proche sémantiquement du texte à analyser, ainsi qu'un score de similitude, qu'on envoie à notre LLM pour qu'il rende son verdict avec les informations actuelles.

2.4 Nouveau Format de Sortie (Standard JSON v2)

La réponse API est désormais beaucoup plus riche, offrant une transparence totale sur le verdict :

```
{
  "verdict": "Slightly Misleading",
  "confiance": 0.85,
  "catégorie": "Politique",
  "justification": "L'article affirme que X est arrivé, mais les rapports de l'AFP
    ↪ indiquent que l'événement a été reporté.",
}
```

2.5 Flux final

2.5.1 1. Entrée

URL ou texte soumis par l'utilisateur.

2.5.2 2. Récupération

Le système extrait les preuves de la base de données factuelles (ChromaDB).

2.5.3 3. Travail du LLM

Llama 3 analyse le texte utilisateur au regard de l'article de notre base avec lequel il a été fournis.

2.5.4 4. Sortie

Production du JSON structuré.

2.6 Sécurité et Robustesse Technique

L'implémentation logicielle a été renforcée par l'ajout de nouvelles dépendances critiques :

- **Pydantic (Schemas)** : Validation stricte des entrées/sorties via `TextSchema` et `FileSchema` pour éviter les injections de données malformées.
- **Gestion des Timeouts** : Configuration de la résilience pour gérer les temps de réponse plus longs des LLM génératifs (utilisation de `wait_for_model: True`).
- **Normalisation des Environnements** : Utilisation d'un fichier `requirements.txt` hybride combinant des versions figées par uv et des extensions flexibles pour le RAG, garantissant la portabilité du projet.

Partie 3

Conclusion

Références

- Kermani, Arshia, Veronica Perez-Rosas, et Vangelis Metsis. 2025. « A Systematic Evaluation of LLM Strategies for Mental Health Text Analysis : Fine-tuning vs. Prompt Engineering vs. RAG ». Prépublié mars 31. <https://doi.org/10.48550/arXiv.2503.24307>.
- Ovadia, Oded, Menachem Brief, Moshik Mishaeli, et Oren Elisha. 2024. « Fine-Tuning or Retrieval ? Comparing Knowledge Injection in LLMs ». Prépublié janvier 30. <https://doi.org/10.48550/arXiv.2312.05934>.
- Pornprasit, Chanathip, et Chakkrit Tantithamthavorn. 2024. « Fine-Tuning and Prompt Engineering for Large Language Models-based Code Review Automation ». Prépublié juin 17. <https://doi.org/10.48550/arXiv.2402.00905>.
- Shin, Jiho, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, et Hadi Hemmati. 2025. « Prompt Engineering or Fine-Tuning : An Empirical Assessment of LLMs for Code ». Prépublié février 19. <https://doi.org/10.48550/arXiv.2310.10508>.
- Vangibhurathachhi, Srinivasa Kalyan. 2025. « Adapting Large Language Models : A Comparative Study of Prompt Engineering and Fine-Tuning ». *International Journal of Emerging Research in Engineering and Technology* 6 (3) : 9-15. <https://doi.org/10.63282/3050-922X.IJERET-V6I3P102>.